# RINS Task 2 Report

## 1 Introduction

In Task 2, the robot had to combine perception, navigation, and decision logic in order to move through the environment and react to objects, people, and anomalies detected by its sensors. The implementation was divided into several modules, where each module solved one part of the full robotic pipeline and communicated with the others through ROS 2 topics and services.

## 2 Methods

### 2.1 Face Detection

YOLO, short for "You Only Look Once", is a real-time object detection method. Instead of first proposing image regions and then classifying them separately, YOLO processes the whole image in a single forward pass of a neural network. The output is a set of detected objects, where each detection contains a class label, a confidence score, and a bounding box in image coordinates. This makes YOLO suitable for robotic perception tasks because the robot can continuously process camera frames while moving.

In our task, YOLO was used as the first stage of the face detection pipeline. The detector receives RGB camera images and searches for visible faces in the scene. The important output for the rest of the system is not the image itself, but the position of the detected person in the map. Therefore, the bounding box returned by YOLO is used only as an intermediate result: it tells us where in the image the target is located, and the corresponding depth data is then used to estimate the target's 3D position.

#### 2.1.1 Destination Compute

### 2.2 Ring Detection

#### 2.2.1 Destination Compute

Text to be added.

### 2.3 Barrel Detection

#### 2.3.1 Destination Compute

Text to be added.

### 2.4 Anomaly Detection

For anomaly detection we used SuperSimpleNet, a neural network designed for visual anomaly detection. The model produces both an image-level anomaly score and a pixel-level anomaly map. The score tells us whether the inspected object should be treated as defective, while the anomaly map localizes the suspicious regions in the image.

In our case the training was done in a fully supervised setting, because the provided dataset already contained anomaly masks together with the pile images. This means the model did not

only learn whether an image was defective, but also had direct pixel-level supervision for where the defect was located. This was important because the final task required both a classification result and a mask that could be included in the generated report.

The first model was trained on the original dataset. During testing in Gazebo, however, we noticed that the lighting in the simulation made the ceramic piles appear much brighter than they appeared in the dataset. This difference caused a domain shift between the training images and the images seen by the robot. To compensate for this, we reused the already trained model and created an additional augmented dataset where the pile images were brightened. The final model used by the ROS node is the augmented model worked better under the simulated lighting conditions.

#### 2.4.1 Navigating Along Belt

The anomaly task is executed only when the robot is positioned in front of the belt. The movement module contains fixed approach poses for the red and green belts and a fixed number of tiles for each belt. After navigating to the belt, the robot aligns itself with the wall, moves to the required distance from the belt, turns to face along the belt, and then checks each pile one by one. Between checks it drives a fixed step distance to the next pile position while correcting its lateral distance from the belt. At each fixed pile position the central movement logic calls the anomaly detection service.

### 2.5 Follow Line

Text to be added.

### 2.6 Navigation Around Map and Yellow Line

Text to be added.

### 2.7 Decision Logic

Text to be added.

## 3 Implementation and Integration

### 3.1 ROS Nodes Architecture

Text to be added.

### 3.2 Face Detection

The face detection module is implemented as the ROS 2 node `detect_faces`. It loads the YOLO model `yolov8n.pt` and subscribes to two synchronized OAK-D camera streams: the RGB image topic `/oakd/rgb/preview/image_raw` and the depth point cloud topic `/oakd/rgb/preview/depth/points`. The synchronization is needed because the RGB frame and the point cloud must describe approximately the same moment in time.

For every synchronized RGB/depth pair, the RGB image is converted to an OpenCV image and passed through YOLO. The model returns bounding boxes for detected targets. For each bounding box, the node computes the center pixel of the box. This center is used as the representative image point of the detected face. The same pixel location is then looked up in the organized point cloud, which gives a 3D point in the camera frame.

The raw 3D point is still expressed relative to the camera, so it cannot be directly used by the navigation system. To integrate the detection with the global map, the node wraps the point

into a `PoseStamped` message using the point cloud frame and transforms it into the `map` frame with TF2. After this transform, the module has the estimated global `x` and `y` position of the detected person.

To make the detection more robust, the node does not immediately publish every single YOLO detection. New detections are first stored as potential faces. If a detected position is seen repeatedly near the same map location, the module updates a running average. Only after the same target has been observed more than the acceptance threshold is it accepted as a real face and assigned an id. Already accepted targets can also be updated and republished if their averaged position changes enough.

Accepted detections are published on the `/face_detect` topic using the custom `FaceDetect` message. The message contains the detected target id and its map position:

```
float32 x
float32 y
int8 id
```

The movement module subscribes to `/face_detect`. When it receives a new id, it adds the person to the pending people queue and visualizes the position. If an already known id is published again with an updated position, the movement module updates that person's stored position and can recompute the navigation destination if the person is currently the active target.

### 3.3 Ring Detection

Text to be added.

### 3.4 Barrel Detection

Text to be added.

### 3.5 Anomaly Detection

The anomaly detection module is implemented as the ROS 2 node `detect_anomalies`. Unlike the face detector, it does not continuously publish detections. Instead, it exposes the service `/detect_anomalies`. This service is called by the central movement module only when the robot has reached the correct fixed position in front of a pile.

The node subscribes to the top camera image topic `/top_camera/rgb/preview/image_raw` and always keeps the latest received frame. When the service is called, the node waits for an available frame, converts it to an OpenCV BGR image, and runs the pile extraction and anomaly detection pipeline on that image.

The first step is to find the pile in the top camera image. The implementation converts the image to grayscale, optionally applies Gaussian blur, and then creates a binary mask using a fixed intensity threshold. After thresholding, a morphological closing operator with an elliptical kernel is applied to fill small holes and connect nearby bright regions. A border strip is removed from the mask to avoid selecting objects at the image edge.

The pile corners are then estimated from the binary mask. The node finds the external contours and selects the largest contour above a minimum area. It computes the convex hull of that contour and tries to approximate it with four points using `approxPolyDP` with increasing approximation tolerances.

After the four pile corners are known, the module uses a homography to normalize the pile image. The source quadrilateral from the camera image is mapped to a square output image with `getPerspectiveTransform` and `warpPerspective`. This gives the anomaly model a consistent square view of the pile, independent of the viewing angle in the top camera. Before inference, the detector also normalizes image brightness in LAB color space.

The warped pile image is passed to the SuperSimpleNet inference wrapper. The model returns an anomaly score, a binary anomaly mask, and a continuous anomaly map. If the score is above the configured threshold, the service returns `defected`; otherwise it returns `not_defected`. If the pile cannot be found or inference fails, the service returns `not_found`. The service response also includes paths to saved images: the warped pile image and the generated anomaly mask. The movement module stores this result for the corresponding tile, and the report-generation module can later include the saved image and mask for defective tiles.

### 3.6 Other Services and Topics

Text to be added.

## 4 Results

Text to be added.

## 5 Division of Work

### 5.1 Who Did What

Text to be added.

### 5.2 Percentages

Text to be added.

## 6 Conclusion

Text to be added.